

Seven Seas A-8

Security Audit

June 4, 2024

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team on May 29th to May 30th 2024.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Low	1	-	-	1
Code Quality	4	-	-	4
Informational	1	1	-	-

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Trust Assumptions:

This update added performance fees to the vault based on the highest share price, or high water mark. It is setup so that a percentage of the increase in share price above the last highest price is taken as a performance fee. The intent is to incentivize the managers of the vault to always increase the share price, benefiting holders. However, since the share price can be set, and high water mark reset by the owners, users must trust that the managers will only use these privileged functions to benefit the protocol as a whole, and not extract unearned performance fees.

Withdrawals via DelayedWithdrawals, can be paused. It is trusted that withdrawals will only be paused when necessary, and to benefit the users. and protocol.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [boring-vault](#)
- **Commit Hash:** 938478e59619c693150a2290ff677191b19a0948

Specifically, we audited the following contracts within this repository.

Source Code	SHA256
src/base/Roles/AccountantWithRateProviders.sol	7cc947b8a8c011d88db1cb3810d4f11bf433902a9f3e96794037ca65ec17ea9c
src/base/Roles/DelayedWithdraw.sol	463e49716872b2726bdf520d74217a0e9c11db60983b8de835706dc80dd753f9
src/base/Roles/TellerWithMultiAssetSupport.sol	57f8f87f1fec7eae4cbe225bf43feb54d87b8c98ce8ff6708be0311b7b8c5e
src/base/Roles/TellerWithRemediation.sol	8f0beba89fd76c4e40c8b5db44d1a417061914cda72b6487b7fa2c7eafd7869a
src/base/DecodersAndSanitizers/Protocols/ITB/eigen_layer/EigenLayerDecoderAndSanitizer.sol	281066bcbcd6d8debbc142c71ab8575ef114390544fa18711964d676b95be124

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

I-1 Resetting high water mark can effect management fees calculations

Q-1 Users have no control on when their withdrawal is executed

Q-2 Users cannot define their own max loss for withdraw requests

Q-3 Cannot change Eigen Layer delegate

Q-4 Withdrawals cannot be paused

I-1 Users can leverage the delayed withdrawal to mitigate risk

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details



Resetting high water mark can effect management fees calculations

TOPIC	STATUS	IMPACT	LIKELIHOOD
Incentive Design	Fixed 	Low	Low

In `AccountantWithRateProviders.sol`'s `resetHighWaterMark()`, the owner can reset the `highwaterMark` used to calculate performance fees. This is intended to only be done to incentivize growth if too far below the last high water mark, and should be used sparingly. However, when it is done it also sets the `totalSharesLastUpdate` to the vaults current shares. `totalSharesLastUpdate` is used in both the performance fees and management fees calculation, so updating this will effect both. In the case of management fees, it is calculated by taking the minimum of the `totalSharesLastUpdate` and current shares, and taking a percent based on time since last update and the management fee. If the `totalSharesLastUpdate` increased or decreased since the call to `updateExchangeRate()`, resetting the high water mark could result in more or less management fees from being taken the next time the exchange rate is updated.

Remediations to Consider

When `resetHighWaterMark()` is called, handle the management fees as well using the current exchange rate, or alternatively do not update `totalSharesLastUpdate`.

Q-1

Users have no control on when their withdrawal is executed

TOPIC

Quality of life

STATUS

Fixed [↗](#)

QUALITY IMPACT

Medium

In `DelayedWithdraw.sol`, withdrawal request can be completed by anyone after the delay has elapsed. The user then has no control of when the withdrawal goes through, and could be executed at a bad time, or a rate they don't like. Allowing users to have more control over their withdrawals if they like may give users a better experience.

Remediations to Consider

Allow users to set if anyone can complete the withdrawal or not. Consider also adding a withdrawal window to prevent long duration pending withdrawals from being used to mitigate risk as mentioned in issue I-1.

Q-2

Users cannot define their own max loss for withdraw requests

TOPIC

Quality of life

STATUS

Fixed [↗](#)

QUALITY IMPACT

Medium

When completing a withdrawal in `DelayedWithdraw.sol`'s `completeWithdraw()` there is a check to ensure that the resulting rate does not exceed the `maxLoss` for the given withdrawal asset. This `maxLoss` value effectively sets a slippage bound on the rate as it changes from request to completion. `maxLoss` is set by the owner via `changeMaxLoss()`, which could be adjusted while withdrawals are requested, leading to undesirable outcomes for users. Depending on how it is adjusted it could lead to allowing users to receive less assets than they expected on initial request, or prevent a withdrawal they really need to execute. If users were allowed to set their own `maxLoss`, they could set

it based on their own needs giving more control to users, and reducing trust requirements.

Remediations to Consider

Allow users to set their own `maxLoss` , and have the defined `maxLoss` per asset be the default in none is set by the user.

Cannot change Eigen Layer delegate

TOPIC	STATUS	QUALITY IMPACT
Protocol Design	Fixed 	High

In `EigenLayerDecoderAndSanitizer.sol` is intended to be used with a Into the Block position manager contract. This position manager is private, but it's `delegate()` function calls `delegateTo()` on the set Eigen Layer `DelegationManager.sol` :

```
function delegateTo(
    address operator,
    SignatureWithExpiry memory approverSignatureAndExpiry,
    bytes32 approverSalt
) external {
    require(!isDelegated(msg.sender), "DelegationManager.delegateTo: staker is
    require(isOperator(operator), "DelegationManager.delegateTo: operator is not
    // go through the internal delegation flow, checking the `approverSignature
    _delegate(msg.sender, operator, approverSignatureAndExpiry, approverSalt);
}
```

However, if the sender already has a delegate, the call reverts. When the Position Manager is deployed a delegate is set, meaning any additional calls to `delegate()` revert. Currently there is no way to undelegate the current delegate leading to no way to change the delegate in the Position Manager after deployment. If a new delegate

were required, a new position manager would need to be deployed, assets migrated after withdrawing, and new merkle leaves having to be set, which would not be ideal.

Remediations to Consider

Add a method to Into the Blocks position manager to allow for undelegation, and add this function to the `EigenLayerDecoderAndSanitizer.sol` contract to allow the vault to call it.

Q-4

Withdrawals cannot be paused

TOPIC	STATUS	QUALITY IMPACT
Protocol Design	Fixed 	Medium

There could be multiple instances where the vault may not want to allow for withdrawals for a period of time. The is by because of increased volatility like a black swan event, or to prevent a security vulnerability from being taken advantage of.

Currently withdrawals can be prevented from completing by removing authorization for the Delayed Withdrawal to call `exit()` on the vault, but a more explicit pausing functionality would be more clean and explicit.

Remediations to Consider

Add pause and unpause permissioned functions and only allow withdrawal requests and completing withdrawals if the contact isn't paused.

I-1

Users can leverage the delayed withdrawal to mitigate risk

TOPIC	STATUS	IMPACT
-------	--------	--------

DelayedWithdraw.sol allows users to request a withdrawal of their vault shares, which initiates a delay before the withdrawal can be completed, and the current value of the shares is noted. However after this delay, they can withdraw their assets out at any time taking the worse of the rate at withdrawal or the rate at request, provided the vault has enough assets to cover it. In the case where there is a black swan event occurs and assets in the cellar are worth substantially less, if a advanced user already had a prepared withdrawal, they could withdrawal in response to this news, and before a new rate has been set, this would allow them to immediately withdraw at the rate of the initial request, dodging the potential losses.

The Q-1 issue's resolution allows a user to prevent anyone from withdrawing their shares, so that gives more ability to leverage this, but a withdrawal window is now limits the duration a withdrawal can be open for. This withdrawal window does not completely disincentives users from attempting this, but it does reduce it's effectiveness.

RESPONSE BY SEVEN SEAS

If users do try to take advantage of this, we have several options available, we can increase the delay time withdraws take to be valid, decrease the completion window, add a withdraw fee for the asset the user is trying to withdraw, then autofill their withdraw once it is valid, and in a worst case scenario we could always turn off withdraws from this contract, and only allow withdraws through the atomic queue.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.